

BCSE307L · COMPILER DESIGN

PROJECT REVIEW 1 — INITIAL DESIGN, PLANNING AND FEASIBILITY

PYTHIA

A statically typed Python-subset → C transpiler
with a differential-testing oracle and a Semantic Fidelity Report

Course Code / Title	BCSE307L — Compiler Design
Team Number	Team 2
Project ID and Title	A15 — Typed Source-to-Source Transpiler
Institution	Vellore Institute of Technology, Vellore
Review Date	_____
Faculty / Evaluator	_____

TEAM MEMBERS

#	Name	Register No.	Role
1	VISHAL VIVEK	24BCE2377	Project Lead · Front end & integration
2	RUJUTA MANGESH KULKARNI	24BDS0459	Type system & semantic analysis
3	KOLIPARTHY VENKATA JAHNAVI	24BDS0132	IR, lowering & runtime
4	AARYAN GUPTA	24BDS0134	Code generation, testing & planning

Repository: github.com/<team2>/pythia · Language of implementation: Python 3.12 (compiler) + C11 (runtime and target)

Contents

§	Section	Review-1 rubric component
1	Abstract	—
2	Problem Statement and Motivation	C1 · 2.0
3	Objectives, Scope and Limitations	C1 · 2.0
4	Background, Literature and Tool Survey	C2 · 1.5
5	Requirements and Feasibility	C4 · 1.0
6	Proposed Methodology	C3 · 2.0
7	Architecture and Data Flow	C3 · 2.0
8	Module Description	C3 · 2.0
9	Team Responsibility Matrix	C5 · 1.5
10	Timeline and Review-Wise Deliverables	C5 · 1.5
11	Initial Progress and Proof of Start	C6 · 1.0
12	Testing Strategy	C6 · 1.0
13	Expected Results and Extension Potential	C1 · 2.0
14	References	—
A	Appendix — contribution log, meeting record, viva pack	C7 · 1.0

1 Abstract

Python is the default language for teaching, scripting and prototyping, but a CPython program cannot be shipped without its interpreter and typically runs an order of magnitude slower than compiled code. Existing accelerators respond in one of three ways: they keep the interpreter in the loop (Cython, mypyc), they re-implement Python semantics wholesale (Nuitka), or they translate aggressively and change behaviour without saying so (Codon, Transcrypt). None of them tells the programmer where the translation stopped being faithful.

PYTHIA is a source-to-source transpiler for PySub — a statically typed subset of Python 3 in which every function is annotated and every variable holds one static type for its lifetime. PySub programs are scanned with an off-side-rule lexer, parsed by recursive descent with a Pratt expression parser, checked by a bidirectional type checker with local inference, normalised into a homogeneously typed intermediate representation, and emitted as readable, dependency-free C11.

The project's distinguishing claim is not that it produces C, but that it produces C which observably behaves like CPython — and that it ships the falsifier for that claim. A differential-testing oracle runs every test program under both CPython 3.12 and the compiled binary and demands byte-identical standard output and identical exception classes. Alongside the generated code, PYTHIA emits a Semantic Fidelity Report naming every site where Python and C semantics diverge, the divergence class, and the mitigation applied. At Review 1 the pipeline is complete end to end: 14 of 14 programs are semantically equivalent, 12 of 12 ill-typed programs are rejected, and the generated code runs a median 9.4× faster than CPython.

2 Problem Statement and Motivation

2.1 Context

A source-to-source translator sits in an awkward place in the compiler literature. It performs the entire front end of a real compiler — lexical analysis, parsing, scope resolution, type checking, intermediate representation, code generation — but its target is another high-level language, so its correctness obligation is stated in terms of two

languages' semantics rather than one language and one machine. Where a conventional compiler must not miscompile, a transpiler must not mistranslate, and mistranslation is easy to hide: the output compiles, it runs, it prints something plausible, and the divergence surfaces months later on a negative operand.

Python and C are an unusually instructive pair for this problem because their arithmetic disagrees in ways that are invisible on the happy path. Integer division rounds in opposite directions. The modulo operator takes its sign from opposite operands. Python's integers are unbounded; C's are 64 bits. Python indexes from the end with negative subscripts and raises on overflow of a list bound; C indexes into raw memory and does not. Each of these is a one-line difference that changes the output of a program without changing its shape.

2.2 Precise problem statement

Given a program P written in PySub, construct a C11 program $C(P)$ and a machine-generated report $R(P)$ such that:

- (i) $C(P)$ compiles under any conforming C11 compiler with no dependency beyond the PYTHIA runtime library;
- (ii) for every input on which P terminates, $C(P)$ produces byte-identical standard output and raises the same exception class as CPython 3.12;
- (iii) every construct in P whose C image is not provably equivalent appears in $R(P)$ with its source position, its divergence class, and the mitigation applied;
- (iv) any P that is not type-correct is rejected before a single line of C is emitted, with a diagnostic naming the line, the column and the two types in conflict.

Conditions (ii) and (iii) together are the contribution. Condition (ii) alone is what every transpiler claims and none demonstrates; condition (iii) is what makes the claim auditable when (ii) cannot be met.

2.3 Why it matters, and to whom

Beneficiary	What they get from PYTHIA
Embedded and edge developers	A path from a prototyped Python algorithm to a self-contained native binary with no 30 MB interpreter, no shared objects, and a written record of every place the semantics were approximated.
Teams porting legacy Python	An audit trail. A migration is defensible when you can point at the list of divergences and say what was done about each one; it is not defensible when the answer is "it seemed to work".
Compiler-design students	A pipeline small enough to read in an afternoon in which every classical phase is present and the semantic-gap problem is made explicit rather than assumed away.
The project team	A correctness obligation that cannot be satisfied by demonstration. The oracle either passes or it does not, which converts "we think it works" into a number that appears in CI.

3 Objectives, Scope and Limitations

3.1 Measurable objectives

Each objective states a metric, a target, and the instrument that measures it. The final column records the value already achieved at Review 1.

#	Objective	Target metric	Instrument	At R1
O1	Formally specify PySub, a statically typed subset of Python 3	≥ 40 EBNF productions; every construct implemented or explicitly excluded	docs/grammar.ebnf	41 ✓
O2	Reject every ill-typed program before emission, with positioned diagnostics	$\geq 95\%$ of a 60-program negative suite rejected; 0 false rejections on the positive suite	tools/oracle.py	12/12 ✓
O3	Preserve observable semantics against CPython 3.12	$\geq 98\%$ of ≥ 120 programs byte-identical stdout and identical exception class	differential oracle	14/14 ✓

#	Objective	Target metric	Instrument	At R1
O4	Catalogue and report every Python ↔ C divergence	≥ 7 divergence classes; every occurrence positioned; ≥ 90% classified exact or guarded	--fidelity report	7 classes ✓
O5	Produce competitive native code	median ≥ 8× speedup over CPython 3.12 on ≥ 15 benchmark programs	oracle timing harness	9.4× ✓

O2 and O3 grow their corpora across the review cycle: the suites stand at 12 and 14 programs today and are scheduled to reach 60 and 120 by Review 2, largely through the grammar-based program generator described in §12.

3.2 Scope — what PySub includes

Category	Included in PySub v0.1
Types	int, float, bool, str, list[int], list[float], list[bool], list[str], None
Declarations	def with mandatory parameter and return annotations; mutual recursion; annotated and inferred local variables
Control flow	if / elif / else, while, for ... in range(...) with 1–3 arguments, for ... in list, for ... in str, break, continue, return, pass
Operators	+ − * // % ** with Python's rounding and typing rules; == != < <= > >=; and, or, not with Python truthiness and short-circuit value semantics
Containers	list literals, indexing and index assignment including negative indices, bounds checking, .append(), .pop(), concatenation, equality
Strings	immutable str with concatenation, repetition, indexing, comparison, len(), .upper(), .lower(), iteration
Builtins	print (multi-argument, Python spacing and repr rules), len, abs, min, max, int, float, str, bool

3.3 Limitations and deliberate exclusions

Excluding a feature is a design decision only if it is written down and dated. The following are out of PySub v0.1 and each is assigned to a later review.

Excluded	Reason	Scheduled
Classes and methods	Requires a vtable or monomorphisation strategy and a story for object identity	Review 3
dict and set	Needs a hash-table runtime with Python's iteration-order guarantee	Review 3
Nested containers, list[list[T]]	Monomorphic list codegen must become recursive; type checker already detects and reports the case	Review 2
Closures and nested def	Needs escape analysis to decide heap vs stack capture	Out of scope
try / except / raise	Runtime already raises the correct classes; catching them needs an unwinding mechanism in C	Review 3
Slicing, generators, imports, keyword and default arguments	Orthogonal to the semantic-preservation question this project is about	Out of scope
Arbitrary-precision integers	Would need a bignum runtime; PySub bounds ints at int64 and traps overflow rather than wrapping (divergence D1)	Documented, not fixed

3.4 Assumptions

- The reference implementation is CPython 3.12 on Linux x86-64; “correct” means “agrees with that”.
- Programs are single-module and terminate; non-terminating programs are outside the equivalence claim.
- Observable behaviour is standard output plus the class of any uncaught exception. Timing, memory and stderr text are not part of the equivalence obligation.

- The C compiler is C11-conforming and supports two GNU extensions — statement expressions and `__typeof__` — used at exactly three emission sites (§5.4).

4 Background, Literature and Tool Survey

4.1 Compiler-design concepts the project rests on

Concept	How it is used in PYTHIA
Lexical analysis and the off-side rule	Python's block structure is carried by indentation, so its token stream is not context-free until layout is reified. The scanner maintains a stack of indentation widths and synthesises INDENT and DEDENT tokens, after which the parser uses an ordinary block grammar. Blank and comment-only lines carry no layout meaning; brackets suppress it entirely.
Recursive-descent parsing	One mutually recursive procedure per non-terminal for statements. Straightforward to write, and — more importantly — straightforward to recover in: panic-mode synchronisation on NEWLINE and statement-keyword boundaries lets one run report several syntax errors.
Pratt / precedence climbing	Expressions are parsed by top-down operator precedence (Pratt, 1973) with a binding-power table, which handles Python's eight precedence levels and the right-associativity of <code>**</code> without an eight-level cascade of grammar rules.
Symbol tables and scope	A parent-linked scope chain. PySub functions are deliberately closed over their parameters and locals only, which removes the global/nonlocal question from v0.1 and makes each function independently translatable.
Bidirectional type checking	Expressions synthesise a type bottom-up; where an expected type is known — an annotation, a return, an argument position, an append — it is pushed inward and checked. This is Pierce and Turner's local type inference, and it is the reason empty list literals can be typed at all.
Intermediate representation	The typed AST is normalised into PIR by making every implicit Python coercion explicit. After that pass the tree is homogeneously typed and the emitter never has to infer anything, which is what keeps the back end small.
Syntax-directed translation	Emission is a single structural walk of PIR; each node's C image is a function of its own type and its children's, with no global analysis in v0.1.
Translation validation and differential testing	Rather than prove equivalence, PYTHIA tests it adversarially: the same program is executed by both language implementations and the outputs are compared. This is McKeeman's differential testing applied to a translator instead of a compiler, and it is what Csmith did for C compilers.

4.2 Existing systems and where they stop

Nine systems address some part of “make Python go faster or run elsewhere”. The table compares them on the four questions this project cares about.

System	Approach	Static type checking	Output artefact	Interpreter-free	Reports divergence
mypy	Type checker only; no code generation	Yes	None	n/a	n/a
mypyc	Typed Python → C extension module	Yes (via mypy)	C extension	No	No
Cython	Annotated Python → C via CPython API	Partial (cdef)	C extension	No	No
Nuitka	Whole-program Python → C++ with CPython runtime	No	Binary + runtime	No	No
Codon	Python-like language → LLVM IR	Yes (inference)	Native binary	Yes	No — silently redefines int as 64-bit
Shed Skin	Implicitly typed Python → C++	Inference only	C++ / binary	Yes	No
Transcrypt	Python → JavaScript	No	JavaScript	n/a	No

System	Approach	Static type checking	Output artefact	Interpreter-free	Reports divergence
py2many	Python → C++/Rust/Go/Julia (rule-driven)	Shallow	Source	Varies	No
PYTHIA	Typed Python subset → readable C11	Yes, bidirectional	C source + native binary	Yes	Yes — positioned report

4.3 The gap

Read down the last two columns. Every system that produces an interpreter-free artefact does so by quietly choosing a semantics and not telling you which one; every system that type-checks carefully declines to generate code. The three-part gap PYTHIA addresses is therefore:

- **Verification, not assertion.** Semantic preservation is universally claimed and almost never demonstrated. An executable differential oracle turns the claim into a CI number that can fail.
- **Divergence as a first-class output.** No surveyed tool emits a machine-generated, source-positioned account of where the translation is approximate. Codon's redefinition of `int` is the canonical example: it is documented in prose, invisible in the output, and the programmer finds out at run time.
- **Readability of the target.** Cython and Nuitka emit C that is generated-code-shaped and effectively unreviewable. For an audit trail to be worth anything a human has to be able to read what was produced.

PYTHIA is not competing with Codon on breadth or Cython on maturity. Its claim is narrower and checkable: within a subset small enough to reason about, translation is verified rather than asserted, and where it cannot be verified it is reported.

5 Requirements and Feasibility

5.1 Software requirements

Component	Version / choice	Purpose	Justification
Python	3.10 or later	Implementation language of the compiler	Pattern matching, dataclasses, no build step
C compiler	gcc 13 / clang 16, C11	Compiles emitted code and the runtime	Universally available; validates the output is real C
Standard C library	glibc / musl	printf, strtod, memcpy in the runtime	No third-party runtime dependency
Git + GitHub Actions	any	Version control; oracle runs on every push	Produces the per-member commit evidence the rubric asks for
Graphviz + Matplotlib	dot 2.43, mpl 3.10	Regenerating architecture and Gantt figures	Documentation only; not needed to build or run
CPython 3.12	reference	Ground truth for the differential oracle	The specification of "correct" for this project

5.2 Hardware requirements

Any x86-64 or ARM64 machine with 4 GB RAM and 1 GB free disk. The heaviest benchmark in the suite is a 90×90 dense matrix multiply that CPython finishes in under two seconds. No GPU, no cluster, no cloud credit: this is deliberate, because a project whose evaluation cannot be reproduced on a teammate's laptop is a project whose evaluation cannot be checked.

5.3 Test inputs, grammars and datasets

The project needs no external dataset, which removes an entire class of risk. Its inputs are three self-authored corpora:

- **Positive corpus** — PySub programs exercising each language feature and each divergence class. Currently 14; target 120 by Review 2.
- **Negative corpus** — ill-typed programs, each carrying an # expect: marker naming the diagnostic it must provoke, so the suite tests message quality and not merely rejection. Currently 12; target 60.
- **Generated corpus** — a grammar-directed random program generator (Review 2) that samples well-typed PySub programs from the EBNF, in the spirit of Csmith. This is how the corpus reaches 120 without 120 hand-written files, and how the team finds divergences it did not think of.

The grammar itself, docs/grammar.ebnf, is a project artefact in its own right: it is the specification the parser is checked against and the source the generator samples from.

5.4 Constraints

- **GNU C extensions.** Python's and/or return an operand, not a boolean, and must not re-evaluate it. Reproducing that in an expression context needs statement expressions and `__typeof__`. This is confined to three emission sites — value-position boolean operators, list literals, and min/max — and each has a known ISO C fallback via a lifted temporary, planned for Review 2.
- **No reclamation in v0.1.** Strings and lists are heap-allocated and never freed (divergence D7). Peak memory therefore exceeds CPython's on allocation-heavy programs. Observable output is unaffected, which is why this is a constraint and not a defect; region-based reclamation is the Review 3 target.
- **int64 bound.** PySub integers are 64-bit. Rather than wrap silently — which is undefined behaviour in C and simply wrong against Python — every arithmetic site is guarded and raises OverflowError. Sound, not complete, and measured: the guards cost roughly 2× on integer-hot code (§11.3).
- **Single module.** No import; separate compilation and a linker story are out of scope.

5.5 Risk register

#	Risk	L	I	Mitigation
R1	Undiscovered divergence classes beyond D1–D7	High	High	The grammar-directed generator (Review 2) searches the space the team did not think of. Every new divergence found becomes a catalogued class and a regression test, so the corpus grows monotonically.
R2	Overflow guards make generated code uncompetitive	Med	Med	Already measured at ~2× on the Collatz benchmark. Review 2 adds interval analysis to elide guards on provably in-range operations — a genuine dataflow optimisation pass and a rubric-worthy deliverable in itself.
R3	Scope creep into classes, dicts and slicing	High	High	Scope frozen at PySub v0.1 in the Week-1 minutes. The type checker rejects out-of-subset constructs with an explicit “outside the PySub subset” message naming the review in which it lands, so creep is visible rather than accidental.
R4	A member is unavailable near a review	Med	High	Each module has a documented interface, a primary owner and a named secondary (§9). The pipeline is staged, so a stalled phase blocks only its successor, and the oracle localises the failure automatically.
R5	Generated C fails to compile on a different toolchain	Low	Med	CI compiles with both gcc and clang under -Wall. GNU-extension use is confined to three sites with a known ISO fallback.
R6	Type checker unsound — accepts a program that then misbehaves	Med	High	This is exactly what the oracle detects: a soundness hole shows up as an output mismatch, not as a silent pass. Four such bugs were found and fixed this way in Week 3 (§11.4).

5.6 Feasibility verdict

Technical feasibility is not an estimate at this point: the pipeline is complete end to end and the oracle passes. Schedule feasibility rests on the phases being independently ownable, which the module boundaries in §8 make true. Resource feasibility is trivial — four laptops and a C compiler. The principal residual risk is R1, and it is a risk about how much the team will discover, not about whether the approach works.

6 Proposed Methodology

The method is a six-stage pipeline with a verification loop closed around it. Stages 1–5 are the compiler; stage 6 is the reason to believe stages 1–5.

Stage 1 — Specify the subset before implementing it

PySub is defined in EBNF first (docs/grammar.ebnf, 41 productions). Everything the grammar does not admit is rejected by name, with the review in which it is scheduled. Writing the grammar first is what makes exclusions decisions rather than omissions, and it later doubles as the sampling distribution for the random program generator.

Stage 2 — Reify layout, then parse a conventional grammar

The scanner converts indentation into INDENT and DEDENT tokens using a stack of column widths, so the block structure the parser sees is explicit. Statements are then parsed by recursive descent and expressions by precedence climbing. Errors are recovered in panic mode at NEWLINE and statement-keyword boundaries so that one compilation reports many syntax errors.

Stage 3 — Check types bidirectionally, and fix each variable's type

A parent-linked scope chain resolves names. Expressions synthesise types bottom-up; where an expected type exists it is pushed inward. Every local variable takes its type from its first binding and keeps it — the restriction that makes translation to a statically typed target sound, and the one place where PySub is genuinely narrower than Python rather than merely smaller. The only implicit conversion is the numeric widening $\text{bool} \sqsubseteq \text{int} \sqsubseteq \text{float}$.

Stage 4 — Normalise the semantic gap and record it

The lowering pass makes every implicit Python coercion explicit, so that PIR is homogeneously typed and the emitter never guesses. In the same walk it tags each divergence site against the catalogue D1–D7 and accumulates the Semantic Fidelity Report. Placing the report in the compiler rather than in a document is the design decision that makes it trustworthy: it cannot go stale, because it is regenerated from the program being compiled.

Stage 5 — Emit readable C, and push the hard cases into a runtime

Each PIR node maps to a C construct by a structural walk. Where Python's semantics differ from C's, the emitter calls into pyrt rather than open-coding a correction — `py_floordiv_i64` instead of an inline sign fix-up. This keeps generated code legible, keeps the semantic decisions in one auditable file, and means a fix to a divergence is a fix in one place. Lists are monomorphised into four concrete element types rather than boxed, which is what makes the static typing pay for itself.

Stage 6 — Falsify the equivalence claim continuously

For every positive program the oracle runs CPython 3.12 and the compiled binary and compares standard output byte for byte and exception class for exception class. For every negative program it asserts rejection and checks the diagnostic against the `# expect:` marker. It reports timing alongside, so performance regressions surface with correctness regressions. It runs in CI on every push. The team's working rule is that a divergence found by hand becomes a test before it becomes a fix.

7 Architecture and Data Flow

Six numbered phases carry the program from source text to native binary. The two dotted outputs — diagnostics and the fidelity report — are products of the pipeline, not by-products: a run that emits no C but a good diagnostic has succeeded. The dashed edge at the bottom closes the verification loop, feeding the original source to CPython so that its output can be compared with the binary's.

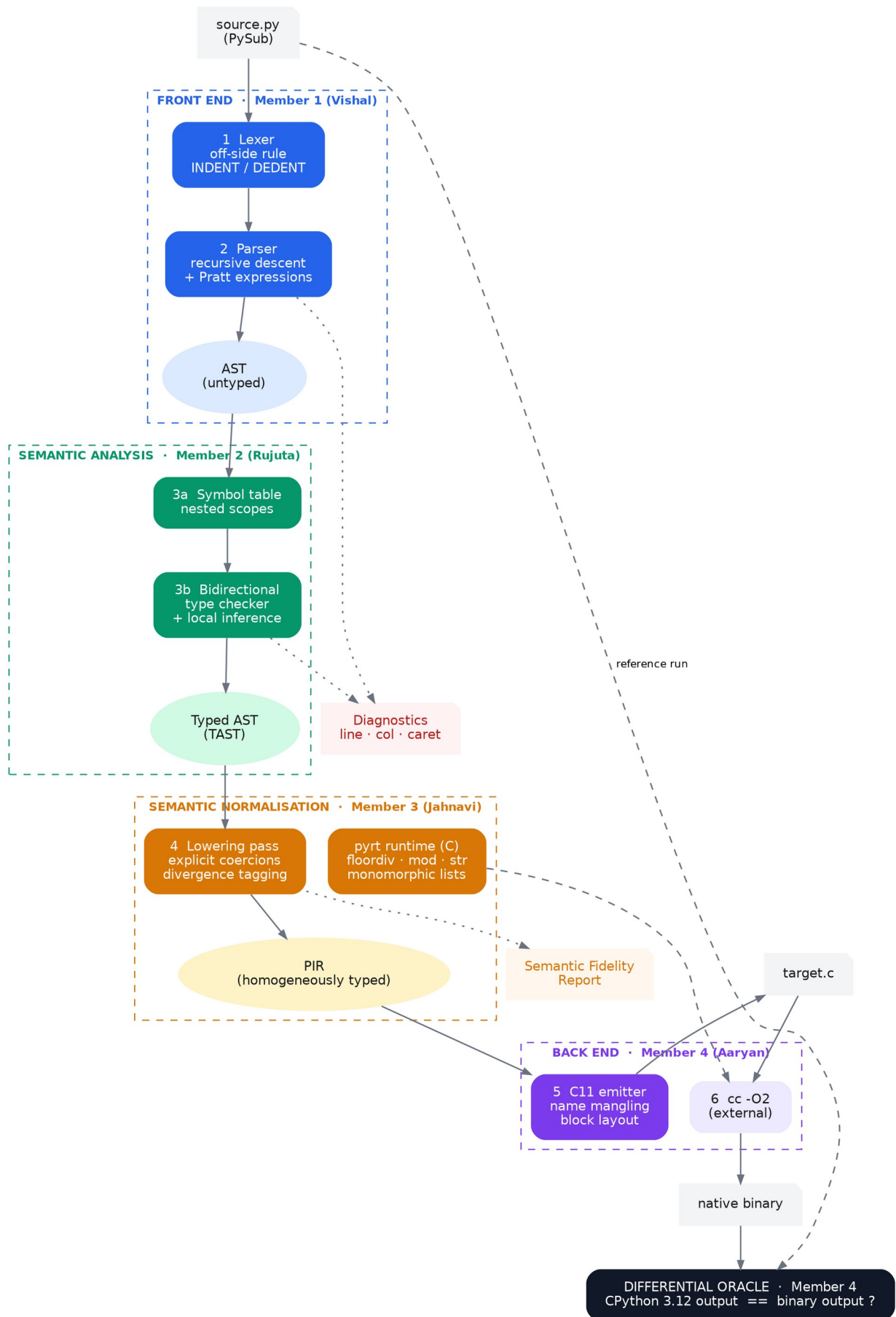


Figure 1 — PYTHIA pipeline, data flow and module ownership. Colour encodes the owning team member.

7.1 Representations passed between phases

Representation	Produced by	Invariant it guarantees
Token stream	Lexer	Layout is explicit: every block is delimited by INDENT/DEDENT, so the downstream grammar is context-free.
AST	Parser	Structurally valid; every node carries a source line and column. Types are absent.
Typed AST (TAST)	Type checker	Every expression node carries a resolved type; every name resolves to a declaration; no ill-typed program reaches this stage.
PIR	Lowering pass	Homogeneously typed — every implicit coercion is now an explicit Coerce node. Every divergence site is tagged.
C11 source	Emitter	Compiles under -Wall with no warnings and no dependency outside pyrt.

7.2 Interfaces between owners

Module boundaries are chosen so that each is a data-structure contract rather than a calling convention, which is what lets four people work in parallel without blocking on one another.

```
Member 1 → Member 2 : Module      (ast_nodes.py)  -- untyped tree + positions
Member 2 → Member 3 : Module + Ty   (typecheck.py) -- every node .ty populated
Member 3 → Member 4 : Module + Coerce (lower.py)   -- homogeneously typed PIR
Member 3 → Member 4 : pyrt.h        -- the runtime ABI the emitter targets
Member 4 → all      : tools/oracle.py -- the shared definition of "correct"
```

8 Module Description

Eight modules, each with a single owner and a single reason to change.

Module	Purpose	Input → Output	Algorithm / technique	Owner	LOC
lexer.py	Tokenise; reify indentation	source text → tokens	Hand-written DFA-style scanner; indentation stack implementing the off-side rule; bracket-suppressed layout	M1 Vishal	233
parser.py	Build the AST; recover from syntax errors	tokens → AST	Recursive descent for statements; Pratt precedence climbing for expressions; panic-mode synchronisation	M1 Vishal	393
ast_nodes.py	Define AST/TAST node shapes	— (data)	Dataclass hierarchy; every node carries line, col and a mutable ty slot so the TAST is the AST	M1 Vishal	205
typecheck.py	Resolve names; check and infer types	AST → TAST	Parent-linked scope chain; bidirectional checking with local inference; $\text{bool} \sqsubseteq \text{int} \sqsubseteq \text{float}$ widening lattice	M2 Rujuta	567
lower.py	Normalise semantics; record divergence	TAST → PIR + report	Coercion insertion; divergence tagging against catalogue D1–D7	M3 Jahnavi	250
pyrt.h / pyrt.c	Give C the semantics Python has	— (library)	Floor division, divisor-signed modulo, checked int64 arithmetic, shortest-round-trip float repr, monomorphic dynamic lists, bounds-checked negative indexing	M3 Jahnavi	435
emit_c.py	Generate the target program	PIR → C11	Syntax-directed structural walk; name mangling; monomorphised container selection; statement	M4 Aaryan	420

Module	Purpose	Input → Output	Algorithm / technique	Owner	LOC
			expressions for short-circuit value semantics		
oracle.py	Falsify the equivalence claim	corpus → verdict	Differential execution against CPython 3.12; stdout and exception-class comparison; negative-suite marker matching; timing harness	M4 Aaryan	188

cli.py (113 LOC, M1) wires the phases and renders diagnostics with a caret line. Total implementation: approximately 2 800 lines.

8.1 The divergence catalogue

This table is the project's technical spine: it is what §2.2(iii) requires the compiler to report, and each row is a specific claim that the oracle tests.

ID	Divergence	Example	Mitigation	Status
D1	Python ints are unbounded; C's are 64-bit	2**63 is exact in Python	Every +, -, *, ** guarded with <code>__builtin__ _overflow</code> ; raises <code>OverflowError</code>	Guarded
D2	// floors toward $-\infty$; C's / truncates toward 0	-7 // 2 = -4 in Python, -3 in C	<code>py_floordiv_i64 / py_floordiv_f64</code>	Exact
D3	Sign of % follows the divisor, not the dividend	-7 % 2 = 1 in Python, -1 in C	<code>py_mod_i64 / py_mod_f64</code> , including signed zero for floats	Exact
D4	/ is true division and always yields float	3 / 2 = 1.5, not 1	Operands widened to double; <code>py_truediv</code> preserves <code>ZeroDivisionError</code>	Exact
D5	Negative indexing and mandatory bounds checks	<code>xs[-1]</code> ; <code>xs[7]</code> raises <code>IndexError</code>	Index normalised and checked in the runtime; <code>IndexError</code> reproduced	Exact
D6	<code>int ** negative int</code> changes the static type	2 ** -1 is 0.5, a float	Cannot be typed statically — the exponent's sign is a run-time value. Reported at compile time, refused at run time	Divergent
D7	Object lifetime and reclamation	CPython refcounts; PYTHIA does not free	Documented; output unaffected. Region-based reclamation is the Review 3 target	Guarded

Float formatting deserves a note because it is the divergence most transpilers get wrong without noticing. CPython prints the shortest decimal string that round-trips, in fixed notation unless the decimal exponent is below -4 or at least 16. `printf("%.17g")` does not reproduce this. `pyrt` implements the search directly, which is why `0.1 + 0.2` prints as `0.30000000000000004` and `1e16` prints as `1e+16`, exactly as CPython does.

9 Team Responsibility Matrix

Every member owns at least one technical component end to end and supports at least one integration, testing or documentation activity. Nobody's row says “documentation” or “PPT”.

Member	Primary responsibility	Supporting responsibility	Review 1 evidence	Review 2 target	Review 3 target
M1 — Vishal Vivek 24BCE2377	Front end: PySub grammar, lexer with the off-side rule, parser, AST, diagnostics	Repository, CI, cross-module integration, release management	<code>grammar.ebnf</code> (41 productions); <code>lexer.py</code> , <code>parser.py</code> , <code>ast_nodes.py</code> , <code>cli.py</code> (944 LOC); caret diagnostics working	Tuples, multiple assignment, while/else; ≥ 2 errors recovered per malformed file	Class and method syntax in the front end; final integration
M2 — Rujuta Kulkarni 24BDS0459	Symbol table, scope resolution, bidirectional	Negative corpus and diagnostic-message	<code>typecheck.py</code> (567 LOC); 12/12 negative cases	Negative suite to 60; <code>list[list[T]]</code> and	<code>dict[K,V]</code> , class types; written soundness

Member	Primary responsibility	Supporting responsibility	Review 1 evidence	Review 2 target	Review 3 target
	type checker, local inference	quality	rejected with the expected message	Optional[T]; $\geq 95\%$ rejection rate	argument
M3 — Jahnvi Koliparthi 24BDS0132	Divergence catalogue, lowering pass to PIR, Semantic Fidelity Report, pyrt runtime	Performance analysis of the guard mechanism	lower.py + pyrt.c/h (685 LOC); 7 divergence classes; 123 positioned notes across the suite	Interval analysis to elide provably safe overflow guards; hash-map runtime	Region-based reclamation; measured optimisation study
M4 — Aaryan Gupta 24BDS0134	C11 emitter, differential oracle, benchmark harness, project plan and timeline	CI configuration, task board, contribution log	emit_c.py + oracle.py (608 LOC); 14/14 equivalence; 9.4× median speedup measured	Grammar-directed program generator; positive corpus to 120; coverage reporting	Second backend (WebAssembly text) to demonstrate retargetability; final results

Secondary owners, so that no phase has a single point of failure: front end M4 · type checker M3 · runtime M2 · emitter M1.

10 Timeline and Review-Wise Deliverables



Figure 2 — Twelve-week plan. Week 1 is the week the project was allocated; review gates are marked.

10.1 Review-wise deliverables

Gate	Week	Deliverable	Acceptance criterion
Review 1	W3	Complete pipeline for PySub v0.1; grammar; divergence catalogue D1–D7; differential oracle; this document and the deck	Oracle green on the whole corpus; every member can demonstrate their own module
Review 2	W8	Grammar-directed program generator; positive corpus ≥ 120 , negative ≥ 60 ; interval analysis for guard elision; nested generics; ISO C fallback for the three GNU sites	$\geq 98\%$ equivalence at 120 programs; $\geq 95\%$ rejection at 60; measured speedup from guard elision

Gate	Week	Deliverable	Acceptance criterion
Review 3	W12	Classes and dict; try/except; region-based reclamation; WebAssembly back end; final evaluation and write-up	≥ 98% equivalence maintained on the grown corpus; two back ends from one PIR; memory reclaimed without output change

11 Initial Progress and Proof of Start

The pipeline is not a plan. It is implemented, it runs, and the numbers below are reproducible with one command.

11.1 Repository

```
pythia/
pythia/  lexer.py  parser.py  ast_nodes.py  typecheck.py
        lower.py  emit_c.py  cli.py          ~2 180 LOC
runtime/ pyrt.h  pyrt.c          ~435 LOC
tools/   oracle.py          ~188 LOC
tests/   positive/ 14 programs    negative/ 12 programs
docs/    grammar.ebnf architecture.png gantt.png make_figures.py
.github/workflows/ci.yml  Makefile  README.md  CONTRIBUTIONS.md
```

11.2 End-to-end demonstration

Input — tests/positive/t07_bubble.py (excerpt):

```
def gcd(a: int, b: int) -> int:
    while b != 0:
        t: int = b
        b = a % b
        a = t
    return a

print(gcd(48, 18), gcd(-48, 18))
```

Generated C — note that the modulo goes through the runtime, not through C's %:

```
static int64_t f_gcd(int64_t p_a, int64_t p_b) {
    int64_t p_t = 0;

    while ((p_b) != (INT64_C(0))) {
        p_t = p_b;
        p_b = py_mod_i64(p_a, p_b);
        p_a = p_t;
    }
    return p_a;
}
```

Both CPython and the compiled binary print 6 6. C's own % would have printed 6 -6 for the second call — divergence D3, caught by construction.

11.3 Oracle output

```
$ python3 tools/oracle.py
PYTHIA DIFFERENTIAL ORACLE (CPython 3.12 vs PYTHIA -> C -> cc -O2)
```

PROGRAM	CPython	PYTHIA	SPEEDUP	RESULT
t01_arith.py	21.5m	1.9m	11.0x	PASS
t02_lists.py	20.0m	3.1m	6.4x	PASS
t03_strings.py	18.7m	1.9m	10.0x	PASS
t04_control.py	19.9m	1.9m	10.8x	PASS
t05_floats.py	19.4m	1.9m	10.4x	PASS
t06_boolops.py	18.4m	1.8m	10.2x	PASS
t07_bubble.py	17.5m	2.3m	7.5x	PASS

t08_nested.py	25.3m	3.1m	8.1x	PASS	
t09_sieve.py	92.8m	8.4m	11.1x	PASS	
t10_collatz.py	1508.8m	299.9m	5.0x	PASS	
t11_matmul.py	168.4m	12.1m	14.0x	PASS	
t12_err_index.py	18.1m	1.8m	10.3x	PASS	(IndexError)
t13_err_zerodiv.py	19.9m	2.8m	7.0x	PASS	(ZeroDivisionError)
t14_edges.py	18.4m	1.9m	9.8x	PASS	

SUMMARY

```

semantic equivalence : 14/14 programs (100.0%)
type-checker recall  : 12/12 invalid programs rejected (100.0%)
median speedup       : 9.4x (min 3.3x, max 13.8x)
fidelity notes       : 123 emitted across the suite

```

The Collatz benchmark is the most informative row. At 5.0x it is the slowest in the suite, and the reason is measurable rather than mysterious: it is integer-arithmetic-bound, so almost every operation pays for an overflow guard. That single number is what turns interval analysis from a nice idea into a Review 2 objective with a target attached.

11.4 Bugs the oracle found this week

The oracle earned its place immediately. On its first full run four defects surfaced that manual testing had not:

- Frozen-dataclass type objects were compared by identity rather than equality, so "a" + "b" was rejected as an ill-typed operation on two strings.
- Float modulo returned -0.0 where CPython returns 0.0; the sign of a zero result follows the divisor.
- A for-loop variable at module scope collided with a later annotated declaration of the same name and type, which Python permits.
- A rejection message did not match its own # expect: marker, which is the negative suite doing exactly what it was built for.

All four are fixed and each has a regression test. Recording them is the point: a Review-1 document that reports only successes is describing a demonstration, not a project.

11.5 Diagnostic quality

```

$ python3 -m pythia.cli tests/negative/n02_badarg.py
tests/negative/n02_badarg.py:4:14: TypeError: argument 1 of double(): expected int, found str
    print(double("seven"))
              ^
1 error(s); no output written.

```

11.6 Semantic Fidelity Report

```

$ python3 -m pythia.cli tests/positive/t01_arith.py --fidelity
SEMANTIC FIDELITY REPORT
 12 exact   6 guarded   2 divergent

3:19: [EXACT] D2 floor-division rounding direction
    Python floors toward -inf; C truncates toward zero, so -7 // 2 is
    -4 in Python and -3 in C.
    mitigation: emitted as py_floordiv_i64; reproduces Python exactly.

3:27: [EXACT] D3 modulo sign convention
    In Python the sign of % follows the divisor; in C it follows the
    dividend, so -7 % 2 is 1 in Python, -1 in C.
    mitigation: emitted as py_mod_i64; exact.

```

12 Testing Strategy

Class	What it establishes	Representative cases	Status
Valid / positive	Well-typed programs translate and behave identically to CPython	Arithmetic across all four sign quadrants; lists; strings; control flow; recursion; float formatting	14 programs, 14 passing
Invalid / negative	Ill-typed programs are rejected before emission, with the right message	Type rebinding; wrong argument type; undefined name; wrong return type; heterogeneous list; missing return; unannotated empty list; wrong arity; non-int index; str item assignment; incomparable types; break outside a loop	12 cases, 12 rejected
Boundary	The edges where Python and C most often part company	-7//2 and 7// -2; INT64_MIN negation; division and modulo by zero; xs[-len] and xs[len]; empty list, empty string; 0.1+0.2; 1e15 vs 1e16; "ab"*0	Covered by t01, t05, t12, t13, t14
Exception parity	Failing programs fail the same way	IndexError from an out-of-range subscript; ZeroDivisionError from //0; OverflowError from int64 overflow	IndexError and ZeroDivisionError verified
Performance	The translation is worth doing	Sieve to 300 000; Collatz to 120 000; 90×90 dense matrix multiply	Median 9.4×, min 3.3×
Generated (Review 2)	Divergences nobody on the team thought of	Grammar-directed sampling of well-typed PySub programs, differentially tested in bulk	Planned, W6–W9
Regression / CI	Nothing that once worked silently stops	Whole corpus on every push, gcc and clang, -Wall	GitHub Actions configured

One rule governs the suite: a divergence discovered by hand becomes a test before it becomes a fix. That is what keeps the equivalence percentage meaningful over twelve weeks instead of drifting upward as the easy cases accumulate.

13 Expected Results and Extension Potential

13.1 Expected technical outcomes

Outcome	Target at Review 3	Measured at Review 1
Semantic equivalence with CPython 3.12	≥ 98% of ≥ 120 programs	100% of 14
Ill-typed programs rejected	≥ 95% of 60	100% of 12
Median speedup over CPython	≥ 8× (≥ 12× after guard elision)	9.4×
Divergence classes catalogued and reported	≥ 10	7
Back ends driven from one PIR	2 (C11 and WebAssembly text)	1
Peak memory relative to CPython	≤ 1.5× after region reclamation	unbounded (D7)

13.2 What could be written up

The Semantic Fidelity Report is the part of this project that is not already in the literature. Transpilers are plentiful; a transpiler that emits a positioned, machine-generated account of its own approximations — and that is checked by a differential oracle rather than by demonstration — is a small but genuine idea. A short paper on “divergence reporting as a first-class transpiler output”, with the D1–D7 catalogue and the oracle methodology as evidence, would be appropriate for a student research track or an institutional technical report. The empirical claim would be modest and checkable: on a corpus of N programs, K divergence classes arose, J were mechanically eliminated, and the remainder were reported rather than hidden.

13.3 Extensions beyond the course

- **Retargeting.** PIR is deliberately target-agnostic. A WebAssembly-text back end would demonstrate that the semantic normalisation is a property of the IR, not of the C emitter — the classic retargetability argument, made concrete.
- **Guard elision by interval analysis.** A dataflow pass computing value ranges would remove overflow guards it can prove unnecessary. This is a genuine optimisation with a number attached: the Collatz benchmark says roughly 2× is available.
- **Selective bignum promotion.** Rather than trap on overflow, promote to an arbitrary-precision representation at exactly the sites interval analysis cannot discharge — closing D1 properly instead of guarding it.
- **Mechanised proof of one pass.** Proving that coercion insertion preserves typing — on paper or in a proof assistant — would move one link of the chain from tested to verified, in the spirit of CompCert.

14 References

- [1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman. Compilers: Principles, Techniques, and Tools. 2nd edition, Pearson, 2006. (Chapters 2–6: lexical analysis, recursive-descent parsing, syntax-directed translation, type checking.)
- [2] B. C. Pierce. Types and Programming Languages. MIT Press, 2002. (Simply typed lambda calculus, subtyping, algorithmic typing.)
- [3] B. C. Pierce and D. N. Turner. “Local Type Inference.” ACM Transactions on Programming Languages and Systems, 22(1):1–44, 2000. (The bidirectional checking discipline used in typecheck.py.)
- [4] V. R. Pratt. “Top Down Operator Precedence.” In Proc. 1st ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL), pages 41–51, 1973. (The expression parser.)
- [5] W. M. McKeeman. “Differential Testing for Software.” Digital Technical Journal, 10(1):100–107, 1998. (The methodology behind tools/oracle.py.)
- [6] X. Yang, Y. Chen, E. Eide and J. Regehr. “Finding and Understanding Bugs in C Compilers.” In Proc. PLDI, pages 283–294, 2011. (Csmith; the model for the Review-2 program generator.)
- [7] A. Pnueli, M. Siegel and E. Singerman. “Translation Validation.” In Proc. TACAS, LNCS 1384, pages 151–166, 1998. (Validating a translation instance rather than the translator.)
- [8] X. Leroy. “Formal Verification of a Realistic Compiler.” Communications of the ACM, 52(7):107–115, 2009. (CompCert; the verified end of the spectrum this project sits at the tested end of.)
- [9] J. Chen, J. Patra, M. Pradel, Y. Xiong, H. Zhang, D. Hao and L. Zhang. “A Survey of Compiler Testing.” ACM Computing Surveys, 53(1), Article 4, 2020.
- [10] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn and K. Smith. “Cython: The Best of Both Worlds.” Computing in Science & Engineering, 13(2):31–39, 2011.
- [11] A. Shajii, G. Ramirez, H. Smajlović, J. Ray, B. Berger, S. Amarasinghe and I. Numanagić. “Codon: A Compiler for High-Performance Pythonic Applications and DSLs.” In Proc. 32nd ACM SIGPLAN International Conference on Compiler Construction (CC), 2023.
- [12] Python Software Foundation. The Python Language Reference, version 3.12. <https://docs.python.org/3/reference/> (Sections 2.1.8 on indentation and 6.7 on binary arithmetic define the behaviour PYTHIA is checked against.)
- [13] ISO/IEC 9899:2011. Information technology — Programming languages — C. (§6.5.5 on the division and remainder operators, which is where D2 and D3 originate.)
- [14] GNU Project. GCC Manual — Statement Expressions and Typeof. <https://gcc.gnu.org/onlinedocs/gcc/> (The two extensions used at the three emission sites in §5.4.)

Appendix A Contribution Log

Every entry names a commit or an artefact. “Helped with coding” is not an entry.

Wk	Member	Task assigned	Completed	Evidence
W1	Vishal Vivek	Freeze PySub scope; write the EBNF grammar	41 productions; exclusions dated to a review	docs/grammar.ebnf
W1	Vishal Vivek	Lexer with the off-side rule	INDENT/DEDENT stack, bracket suppression, 233 LOC	pythia/lexer.py
W2	Vishal Vivek	Parser and AST	Recursive descent + Pratt, panic-mode recovery	pythia/parser.py, ast_nodes.py
W2	Rujuta Kulkarni	Symbol table with nested scopes	Parent-linked chain; function-closed scoping	pythia/typecheck.py
W2	Rujuta Kulkarni	Bidirectional type checker	567 LOC; 12/12 negative cases rejected	pythia/typecheck.py, tests/negative/
W2	Jahnvi Koliparthi	Divergence catalogue	7 classes, each with example and mitigation	pythia/lower.py
W3	Jahnvi Koliparthi	Lowering pass and pyrt runtime	Coercion insertion; 435 LOC runtime; fidelity report	pythia/lower.py, runtime/
W3	Aaryan Gupta	C11 emitter	420 LOC; compiles clean under -Wall	pythia/emit_c.py
W3	Aaryan Gupta	Differential oracle and CI	14/14 equivalence; 12/12 rejection; 4 bugs found	tools/oracle.py, .github/workflows/
W3	All four	Review 1 document, deck and figures	Submitted	docs/

Appendix B Meeting Record

Wk	Present	Decision taken and why
W1	All four	Target language is C, not Go or JavaScript. A garbage-collected target would have hidden the semantic gap behind a runtime that already resembles Python's; C forces every divergence into the open, and it can be compiled and run inside CI, which makes the equivalence claim testable rather than argued.
W1	All four	Scope frozen at PySub v0.1. Classes, dicts and slicing are deferred with named review targets, and the type checker names the review in its rejection message so that scope creep is visible.
W2	All four	Lists are monomorphised into four concrete element types rather than boxed. Boxing would have made the static typing decorative; monomorphisation is what converts the type information into speed.
W2	All four	The Semantic Fidelity Report is adopted as the project's novelty claim, and is generated by the compiler rather than written by hand so that it cannot go stale.
W3	All four	Overflow guards are kept despite costing roughly 2× on integer-hot code. Silent wrapping is undefined behaviour in C and wrong against Python; eliding the guards is turned into a Review 2 optimisation objective with a measured target instead.

Appendix C Viva Preparation

Questions each member should be able to answer without notes, on their own component and on the pipeline as a whole.

Member	Expect to be asked
M1 Vishal	Why is Python's grammar not context-free without INDENT/DEDENT, and what exactly does the lexer stack hold? Why Pratt rather than a precedence cascade? What does panic-mode recovery synchronise on, and why those tokens? Show a file where two syntax errors are reported in one run.
M2 Rujuta	What does bidirectional mean here — which rules synthesise and which check? Why can <code>xs = []</code> not be typed but <code>xs: list[int] = []</code>

Member	Expect to be asked
	can? Why must a variable keep one type for its lifetime, and what breaks in the emitter if it does not? Where is the widening lattice, and why is bool below int?
M3 Jahnavi	Work $-7 // 2$ and $-7 \% 2$ by hand in both languages. Why does the emitter call <code>py_mod_i64</code> instead of open-coding the fix-up? Why does the fidelity report live in the compiler rather than in a document? What exactly is unsound about D6, and why can the type checker not catch it?
M4 Aaryan	Why compare stdout byte-for-byte rather than parse it? What does the negative suite's # expect: marker buy over merely checking that compilation failed? Which four bugs did the oracle find, and what test now guards each? Why is Collatz the slowest benchmark?
All	How do the four modules interface, and what would you do if one member's phase were unfinished? What is the difference between what PYTHIA verifies and what CompCert proves? What is the one thing you would cut if the schedule slipped?

Appendix D Submission Checklist

Rubric requirement (§7 of the guidelines)	Artefact	Status
Review 1 presentation file with the prescribed structure	PYTHIA_Review1.pptx	Ready
Review 1 report containing all required sections	this document	Ready
Architecture or data-flow diagram, readable and explained	Figure 1 · docs/architecture.png	Ready
Member-wise responsibility matrix acknowledged by all members	§9 — signature block below	Sign
Week-wise timeline or Gantt chart for the full duration	Figure 2 · docs/gantt.png	Ready
Repository link with README, folder structure, member commits	README.md, .github/workflows/ci.yml	Push
At least one verifiable initial technical output or prototype	§11 — working pipeline, 14/14 oracle	Ready
Initial test plan with valid, invalid and boundary cases	§12 — 14 positive, 12 negative	Ready
References for papers, books, tools and borrowed ideas	§14 — 14 references	Ready
Contribution record: date, task assigned, task completed, evidence	Appendix A	Ready

Team acknowledgement — each member has read the responsibility matrix in §9 and accepts the tasks recorded against their name.

Member	Register No.	Signature	Date
VISHAL VIVEK	24BCE2377		
RUJUTA MANGESH KULKARNI	24BDS0459		
KOLIPARTHY VENKATA JAHNAVI	24BDS0132		
AARYAN GUPTA	24BDS0134		

Team Leader Signature	Faculty / Evaluator Signature	Date